Construindo o App Botão Logcat

Um guia visual passo a passo sobre a arquitetura de um aplicativo Android utilizando Jetpack Compose e Material 3.

ETEC
zona leste

O resultado final em ação

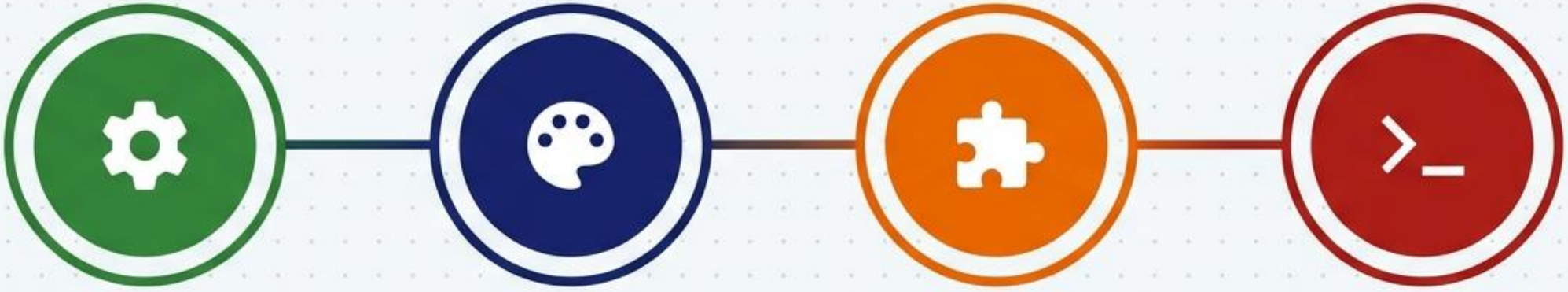
- **Quatro botões semânticos na interface** (ElevatedButton).
- Cada botão **aciona uma severidade diferente** no console de diagnóstico.
- Feedback imediato para o **desenvolvedor separar** rotina de falha crítica.

Terminal Logcat

```
2024-05-17 15:23:13.789 8546-8546 TestAndroid    com.example.introductionjetpack D App: Clicou em DEBUG!
2024-05-17 15:23:16.033 8546-8574 ProfileInstaller com.example.introductionjetpack D Installing profile for
com.example.introductionjetpack
2024-05-17 15:23:25.772 8546-8565 EGL_emulation  com.example.introductionjetpack D app_time_stats:
avg=195.93ms min=3.79ms max=9218.87ms
2024-05-17 15:23:25.813 8546-8546 TestAndroid    com.example.introductionjetpack I App: Clicou em INFO!
```

Desconstruindo as camadas do aplicativo

O desenvolvimento moderno no Android com Compose funciona em camadas modulares. Vamos analisar este projeto de fora para dentro.



Configuração

Gradle e dependências do projeto.

Sistema de Design

Cores, Tipografia e Tema Material 3.

Interface (UI)

Layouts e Componentes Reutilizáveis (Compose)

Lógica

Registro de Eventos e Tratamento de Exceções (Logcat)

Definindo a paleta de cores semânticas

Em Color.kt, além das cores base do Material, o sistema define variáveis específicas para representar os estados do sistema.

```
13  val DarkGreen = Color(0xFF388E3C)
14  val DarkBlue = Color(0xFF1A237E)
15  val DarkOrange = Color(0xFFFF6F00)
16  val DarkRed = Color(0xFFB71C1C)
```

DarkGreen
0xFF388E3C
Ações de depuração.

DarkBlue
0xFF1A237E
Informações gerais.

DarkOrange
0xFFFF6F00
Avisos de atenção.

DarkRed
0xFFB71C1C
Erros críticos.

Padronizando a tipografia com Material 3

O arquivo Type.kt personaliza o estilo global de texto da aplicação. O estilo **bodyLarge** dita a aparência dos textos principais da interface.

```
11    bodyLarge = TextStyle(
12        fontFamily = FontFamily.Default,
13        fontWeight = FontWeight.Normal,
14        fontSize = 16.sp,
15        lineHeight = 24.sp,
16        letterSpacing = 0.5.sp
17    )
```

Tamanho: **16.sp**

Altura da linha: **24.sp**

Espaçamento: **0.5.sp**

Scale-independent Pixels (.sp): Garante que o texto escale perfeitamente com as configurações de acessibilidade do dispositivo do usuário.

ETEC zona leste

Construindo funções de cor customizadas

Com a paleta definida, o Theme.kt cria funções @Composable dedicadas que retornam configurações pré-montadas de ButtonDefaults. Isso garante consistência visual em todo o app.

```
73  @Composable
74  fun DebugButtonColors() = ButtonDefaults.buttonColors(containerColor = DarkGreen)
75
76  @Composable
77  fun WarningButtonColors() = ButtonDefaults.buttonColors(containerColor = DarkOrange)
78
79  @Composable
80  fun ErrorButtonColors() = ButtonDefaults.buttonColors(containerColor = DarkRed)
81
82  @Composable
83  fun InfoButtonColors() = ButtonDefaults.buttonColors(containerColor = DarkBlue)
```

Criando componentes de interface reutilizáveis

Em vez de reescrever o código quatro vezes, encapsulamos um ElevatedButton genérico na função genérica ActionButton.

```
93   @Composable
94   fun ActionButton(
95       text: String,
96       buttonColors: ButtonColors = ButtonDefaults.buttonColors(),
97       modifier: Modifier = Modifier,
98       block: () -> Unit
99   ){
100      ElevatedButton(
101          onClick = block,
102          shape = RoundedCornerShape(5.dp),
103          colors = buttonColors,
104          modifier = modifier
105      ) { this: RowScope
106          Text(text = text)
107      }
108  }
```

Parâmetros Dinâmicos: Recebe o texto, a cor e a função a ser executada (block) como variáveis.

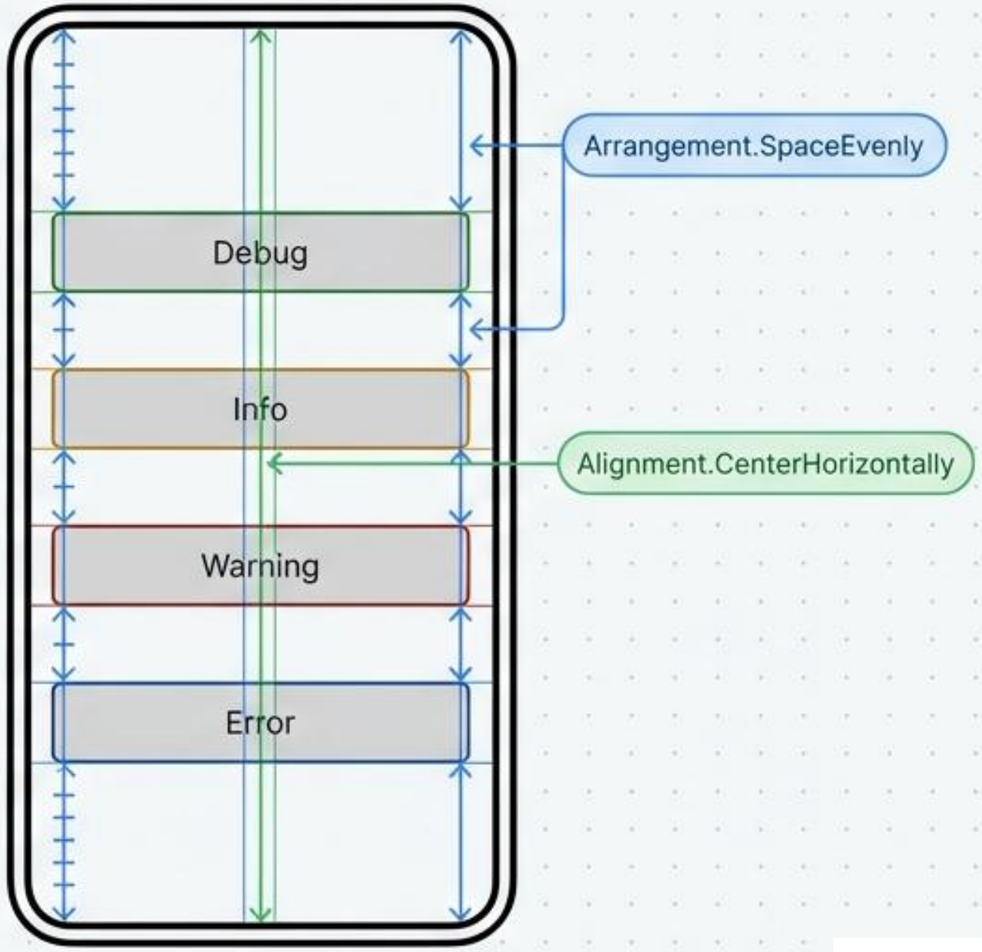
Estilo Base: Aplica cantos arredondados padronizados a todas as instâncias.

ETEC zona leste

Estruturando a tela com layout em coluna

```
51  Column(
52      verticalArrangement = Arrangement.SpaceEvenly,
53      horizontalAlignment = Alignment.CenterHorizontally,
54  ) {  this: ColumnScope
```

A interface principal é montada pela função App() dentro da MainActivity. O componente Column empilha os botões verticalmente com distribuição perfeita de espaço.



Registrando eventos de rotina no Logcat

Cada ActionButton passa um comando específico para a classe utilitária Log do Android, utilizando a tag global 'TestAndroid'.

```
1
2   Log.d(TAG, msg: "App: Clicou em DEBUG!")
3
```

Log.d (Debug): Para informações úteis durante o desenvolvimento.

```
1
2   Log.i(TAG, msg: "App: Clicou em INFO!")
3
```

Log.i (Info): Para mensagens informativas sobre o fluxo normal do app.

Tratando exceções e capturando erros críticos

O aplicativo demonstra resiliência ao forçar um erro através de um bloco try-catch.

```
83      }
84      ActionButton(
85          text = "Error",
86          buttonColors = ErrorButtonColors(),
87          modifier = Modifier.fillMaxWidth( fraction: 0.5f)
88      ) {
89          try {
90              throw RuntimeException("Clicou em Error!")
91          } catch (ex: Exception) {
92              Log.e(TAG, "App: ${ex.message}")
93          }
94      }
```

Log.w (Warning): Indica problemas potenciais que não param a execução.

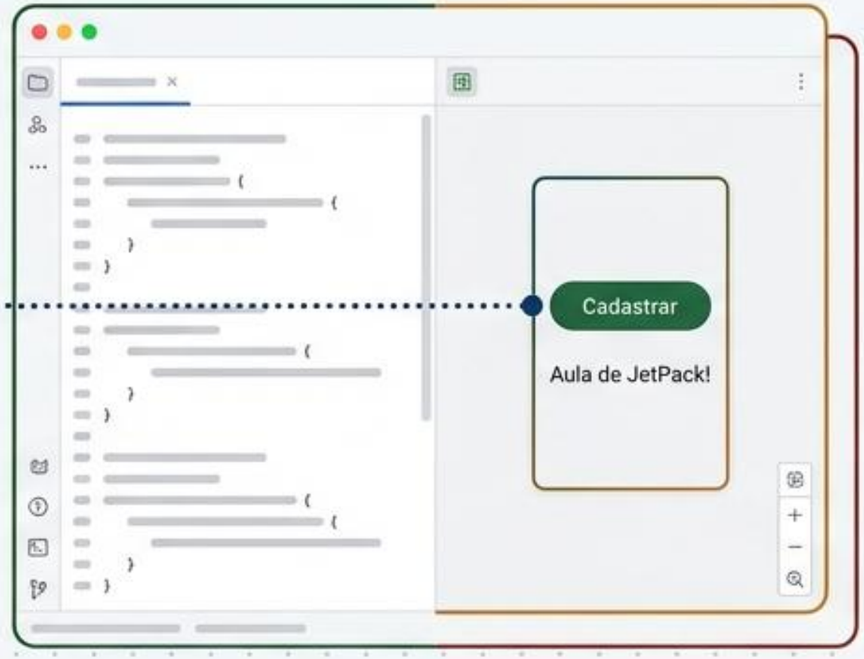
Log.e (Error): O botão simula uma falha no sistema disparando um RuntimeException. O bloco intercepta a falha e registra a mensagem, impedindo que o aplicativo quebre (crash).

ETEC zona leste

Acelerando o desenvolvimento com Previews

A anotação @Preview permite visualizar componentes da interface em tempo real no Android Studio, sem a necessidade de compilar o aplicativo completo em um emulador.

```
121  @Preview(showBackground = true, widthDp = 150, heightDp = 200)
122  @Composable
123  fun AppPreview(){
124      IntroductionJetPackTheme {
125          App()
126      }
127  }
128
129  @Preview(showBackground = true, widthDp = 120)
130  @Composable
131  fun ActionButtonPreview(){
132      ActionButton(text = "Cadastrar") {
133
134      }
135  }
136
137  @Preview(showBackground = true)
138  @Composable
139  fun GreetingPreview() {
140      IntroductionJetPackTheme {
141          Greeting( name: "JetPack")
142      }
143  }
```



O projeto utiliza múltiplos previews para isolar a tela inteira (AppPreview) e componentes individuais (ActionButtonPreview, GreetingPreview).

Configurando os bastidores no Gradle

O ecossistema Compose requer habilitações específicas no arquivo build.gradle.kts no nível do módulo.

```
12        minSdk = 24
13        targetSdk = 33
```

Compatibilidade: Foco em APIs modernas para suportar as bibliotecas do Material 3.

```
40    buildFeatures {
41        compose = true
42    }
```

Habilitação: Instruí o compilador a processar as anotações do Jetpack Compose.

ETEC zona leste

A fundação do desenvolvimento Android moderno

O aplicativo "Botão Logcat" é simples na interface, mas profundo em sua arquitetura. Dominar estes pilares é o primeiro passo para arquiteturas complexas e escaláveis.

Configurações de Build
Gradle

Sistema de Estilização
Tema Material 3

Interface Declarativa
Compose

Monitoramento Crítico
Logcat